# Optimal Classification Trees: Main Results and Additional Studies

March 2026

## 1 Introduction

This report studies the construction of optimal classification trees on seven datasets: `iris`, `seeds`, `wine`, `banknote`, `wdbc`, `titanic`, and `diabetes`. The main objective is to compare the exact formulation with two lighter approaches: a grouped formulation and an iterative method built from the same grouping idea.

Two complementary studies are also included. The first one adds simple variables built from the original columns. The second one uses a dedicated univariate unit-flow formulation to study *opposite corners*, *linked sets*, rounding, and a simple cutting-plane strategy.

## 2 Experimental Setup

All datasets are normalized to the $[0, 1]$ range. The train/test split is fixed at 80/20 with seed 1. The studied depths are $D \in \{2, 3, 4\}$.

When several runs existed for the same script and dataset, the main tables keep only the run with the largest available time limit. In `main_merge.jl` and `main_iterative_algorithm.jl`, several values of $\gamma$ are tested at each depth; to keep the tables readable, only the configuration with the lowest test error is kept for each depth, with train error and then solving time used as tie-breakers.

The scripts `main.jl` and `main_merge.jl` are studied in both the univariate and multivariate settings. The analysis of `main_iterative_algorithm.jl` is limited to the univariate case. For this algorithm, the solver limit applies to each successive subproblem, so the reported total time can exceed 180.0 s. The grouped and iterative tables retained in the final report focus on the five datasets required by the subject: the three provided datasets plus `banknote` and `wdbc`. Short exploratory grouped or iterative runs on `titanic` and `diabetes` were not kept in the final comparison tables. The study of opposite-corner and linked-set equalities uses a separate depth-2 comparison with a 30.0 s limit per LP or MIP solve, because each reported number already combines several solves.

## 3 Main Results

### 3.1 Exact Formulation

The exact formulation provides the reference point for the rest of the study. Tables 1 and 2 gather the results obtained on the seven datasets. Each cell reports train/test errors, then solving time, and finally the solver gap.

| Dataset | $D$ | Univariate | Multivariate |
|---|---|---|---|
| iris | 2 | train/test = 5/1<br>3.4 s, gap 0.0% | train/test = 1/3<br>0.5 s, gap 0.0% |
| iris | 3 | train/test = 0/3<br>26.3 s, gap 0.0% | train/test = 0/3<br>2.0 s, gap 0.0% |
| iris | 4 | train/test = 0/1<br>28.5 s, gap 0.0% | train/test = 0/2<br>6.2 s, gap 0.0% |
| seeds | 2 | train/test = 10/3<br>66.1 s, gap 0.0% | train/test = 0/0<br>0.5 s, gap 0.0% |
| seeds | 3 | train/test = 6/3<br>180.0 s, gap 3.7% | train/test = 0/1<br>3.3 s, gap 0.0% |
| seeds | 4 | train/test = 4/5<br>180.1 s, gap 2.4% | train/test = 0/2<br>16.5 s, gap 0.0% |
| wine | 2 | train/test = 5/2<br>31.4 s, gap 0.0% | train/test = 0/1<br>0.3 s, gap 0.0% |
| wine | 3 | train/test = 0/2<br>56.0 s, gap 0.0% | train/test = 0/0<br>1.1 s, gap 0.0% |
| wine | 4 | train/test = 2/4<br>180.1 s, gap 1.4% | train/test = 0/2<br>9.4 s, gap 0.0% |
| banknote | 2 | train/test = 390/100<br>181.1 s, gap 14.8% | train/test = 0/1<br>3.9 s, gap 0.0% |
| banknote | 3 | train/test = 382/103<br>180.2 s, gap 22.0% | train/test = 0/0<br>6.7 s, gap 0.0% |
| banknote | 4 | train/test = 391/102<br>180.5 s, gap 23.5% | train/test = 605/157<br>181.6 s, gap 123.0% |
| wdbc | 2 | train/test = 20/11<br>180.1 s, gap 4.6% | train/test = 0/6<br>4.2 s, gap 0.0% |
| wdbc | 3 | train/test = 18/14<br>180.3 s, gap 4.1% | train/test = 0/5<br>19.1 s, gap 0.0% |
| wdbc | 4 | train/test = 25/13<br>180.2 s, gap 6.1% | train/test = 169/43<br>181.8 s, gap 59.1% |

Table 1: Results of `main.jl` on `iris`, `seeds`, `wine`, `banknote`, and `wdbc`.

| Dataset | $D$ | Univariate | Multivariate |
|---|---|---|---|
| titanic | 2 | train/test = 137/38<br>360.3 s, gap 23.8% | train/test = 125/39<br>360.8 s, gap 21.3% |
| titanic | 3 | train/test = 118/32<br>360.3 s, gap 19.9% | train/test = 141/38<br>360.3 s, gap 28.5% |
| titanic | 4 | train/test = 163/45<br>361.2 s, gap 29.7% | train/test = 263/79<br>360.7 s, gap 58.6% |
| diabetes | 2 | train/test = 0/0<br>65.2 s, gap 0.0% | train/test = 0/6<br>29.8 s, gap 0.0% |
| diabetes | 3 | train/test = 3880/967<br>361.3 s, gap 4.8e9% | train/test = 0/6<br>19.3 s, gap 0.0% |
| diabetes | 4 | train/test = 2719/679<br>362.5 s, gap 130.7% | train/test = 3880/967<br>362.5 s, gap 421.7% |

Table 2: Results of `main.jl` on `titanic` and `diabetes`.

The main trend is still clear on the first five datasets. On `seeds`, `wine`, and `banknote`, the multivariate formulation clearly dominates at depths 2 and 3, with much shorter solving times and very low, sometimes zero, test errors. On `iris`, the difference is less pronounced, but the multivariate version remains faster. On `wdbc`, it improves test error at depths 2 and 3, then degrades strongly at depth 4. The two larger tabular datasets are more mixed: on `titanic`, both variants remain expensive and the best result changes with depth; on `diabetes`, the reported depth-2 univariate run reaches zero errors, while deeper univariate models become unstable.

### 3.2 Grouped Formulation

The script `main_merge.jl` replaces individual samples with same-class clusters that can no longer be split. Tables 3 and 4 keep only the best value of $\gamma$ at each depth. The univariate and multivariate columns therefore correspond to the best configuration found for each depth.

| Dataset | $D$ | Univariate | Multivariate |
|---|---|---|---|
| iris | 2 | $\gamma = 20\%$<br>train/test = 5/1<br>0.1 s, gap 0.0% | $\gamma = 20\%$<br>train/test = 1/0<br>0.5 s, gap 0.0% |
| iris | 3 | $\gamma = 40\%$<br>train/test = 0/1<br>2.7 s, gap 0.0% | $\gamma = 100\%$<br>train/test = 0/0<br>1.8 s, gap 0.0% |
| iris | 4 | $\gamma = 40\%$<br>train/test = 0/1<br>1.8 s, gap 0.0% | $\gamma = 100\%$<br>train/test = 0/1<br>14.0 s, gap 0.0% |
| seeds | 2 | $\gamma = 60\%$<br>train/test = 12/2<br>2.2 s, gap 0.0% | $\gamma = 40\%$<br>train/test = 0/0<br>0.5 s, gap 0.0% |
| seeds | 3 | $\gamma = 60\%$<br>train/test = 5/2<br>180.0 s, gap 1.8% | $\gamma = 40\%$<br>train/test = 0/1<br>1.9 s, gap 0.0% |
| seeds | 4 | $\gamma = 60\%$<br>train/test = 4/2<br>180.1 s, gap 1.8% | $\gamma = 80\%$<br>train/test = 0/0<br>34.1 s, gap 0.0% |
| wine | 2 | $\gamma = 60\%$<br>train/test = 5/2<br>2.1 s, gap 0.0% | $\gamma = 100\%$<br>train/test = 0/1<br>0.7 s, gap 0.0% |
| wine | 3 | $\gamma = 80\%$<br>train/test = 0/2<br>51.2 s, gap 0.0% | $\gamma = 0\%$<br>train/test = 0/1<br>0.1 s, gap 0.0% |
| wine | 4 | $\gamma = 60\%$<br>train/test = 0/3<br>44.2 s, gap 0.0% | $\gamma = 80\%$<br>train/test = 0/0<br>7.7 s, gap 0.0% |

Table 3: Best configurations of `main_merge.jl` on `iris`, `seeds`, and `wine`.

| Dataset | $D$ | Univariate | Multivariate |
|---|---|---|---|
| banknote | 2 | $\gamma = 40\%$<br>train/test = 399/102<br>59.1 s, gap 0.0% | no feasible solution<br>within 180.0 s |
| banknote | 3 | $\gamma = 20\%$<br>train/test = 414/102<br>19.6 s, gap 0.0% | no feasible solution<br>within 180.0 s |
| banknote | 4 | $\gamma = 0\%$<br>train/test = 492/118<br>1.0 s, gap 0.0% | no feasible solution<br>within 180.0 s |
| wdbc | 2 | $\gamma = 80\%$<br>train/test = 18/6<br>180.1 s, gap 4.1% | $\gamma = 20\%$<br>train/test = 0/6<br>0.6 s, gap 0.0% |
| wdbc | 3 | $\gamma = 80\%$<br>train/test = 16/4<br>523.7 s, gap 3.9% | $\gamma = 0\%$<br>train/test = 0/6<br>1.6 s, gap 0.0% |
| wdbc | 4 | $\gamma = 80\%$<br>train/test = 29/6<br>180.3 s, gap 7.1% | $\gamma = 40\%$<br>train/test = 0/4<br>36.3 s, gap 0.0% |

Table 4: Best configurations of `main_merge.jl` on `banknote` and `wdbc`.

This grouped formulation remains credible in the univariate setting on the three smaller datasets. It

reaches almost the same error levels as the exact univariate approach, often with lower times at depths 2 and 3. In the multivariate setting, it is actually strong on `iris`, `seeds`, `wine`, and `wdbc`: at the retained values of $\gamma$, the grouped trees reach zero training error on all four datasets and often very low test error in a few seconds. The clear exception is `banknote`, where no multivariate grouped run produced a feasible solution within 180.0 s. Since the final grouped comparison is restricted to the five required datasets, the short exploratory grouped runs on `titanic` and `diabetes` are not discussed further here.

### 3.3 Iterative Algorithms

The script `main_iterative_algorithm.jl` is studied only in the univariate case. Tables 5 and 6 compare the best versions of $F_h^S$, $F_h^S$ with *shifts*, and $F_e^S$. Here again, each cell reports train/test errors, total time, and the gap.

| Dataset | $D$ | $F_h^S$ | $F_h^S$ + shifts | $F_e^S$ |
|---|---|---|---|---|
| iris | 2 | $\gamma = 0\%$<br>train/test = 5/1<br>1.8 s, gap 0.0% | $\gamma = 0\%$<br>train/test = 5/1<br>1.5 s, gap 0.0% | $\gamma = 60\%$<br>train/test = 5/1<br>8.9 s, gap 0.0% |
| iris | 3 | $\gamma = 20\%$<br>train/test = 0/1<br>20.8 s, gap 0.0% | $\gamma = 0\%$<br>train/test = 0/0<br>9.0 s, gap 0.0% | $\gamma = 80\%$<br>train/test = 0/3<br>19.5 s, gap 0.0% |
| iris | 4 | $\gamma = 0\%$<br>train/test = 0/1<br>4.4 s, gap 0.0% | $\gamma = 40\%$<br>train/test = 0/0<br>18.5 s, gap 0.0% | $\gamma = 80\%$<br>train/test = 0/1<br>66.7 s, gap 0.0% |
| seeds | 2 | $\gamma = 40\%$<br>train/test = 10/2<br>14.0 s, gap 0.0% | $\gamma = 40\%$<br>train/test = 10/2<br>11.3 s, gap 0.0% | $\gamma = 60\%$<br>train/test = 10/2<br>66.3 s, gap 0.0% |
| seeds | 3 | $\gamma = 80\%$<br>train/test = 7/1<br>180.1 s, gap 2.4% | $\gamma = 40\%$<br>train/test = 0/0<br>180.0 s, gap n/a | $\gamma = 60\%$<br>train/test = 9/2<br>180.1 s, gap 1.8% |
| seeds | 4 | $\gamma = 80\%$<br>train/test = 2/2<br>180.2 s, gap 1.2% | $\gamma = 60\%$<br>train/test = 0/0<br>180.1 s, gap 0.6% | $\gamma = 80\%$<br>train/test = 4/2<br>180.1 s, gap 1.8% |
| wine | 2 | $\gamma = 0\%$<br>train/test = 5/2<br>9.9 s, gap 0.0% | $\gamma = 40\%$<br>train/test = 0/0<br>26.0 s, gap 0.0% | $\gamma = 80\%$<br>train/test = 5/2<br>69.0 s, gap 0.0% |
| wine | 3 | $\gamma = 0\%$<br>train/test = 3/1<br>180.0 s, gap n/a | $\gamma = 20\%$<br>train/test = 0/0<br>29.7 s, gap 0.0% | $\gamma = 80\%$<br>train/test = 0/3<br>22.3 s, gap 0.0% |
| wine | 4 | $\gamma = 20\%$<br>train/test = 0/2<br>21.8 s, gap 0.0% | $\gamma = 20\%$<br>train/test = 0/0<br>36.8 s, gap 0.0% | $\gamma = 80\%$<br>train/test = 14/5<br>180.1 s, gap 1.4% |

Table 5: Best configurations of `main_iterative_algorithm.jl` on `iris`, `seeds`, and `wine`.

| Dataset | $D$ | $F_h^S$ | $F_h^S$ + shifts | $F_e^S$ |
|---|---|---|---|---|
| banknote | 2 | $\gamma = 0\%$<br>train/test = 136/42<br>12.7 s, gap 0.0% | $\gamma = 0\%$<br>train/test = 136/42<br>11.1 s, gap 0.0% | $\gamma = 0\%$<br>train/test = 166/49<br>180.5 s, gap 0.2% |
| banknote | 3 | $\gamma = 0\%$<br>train/test = 149/45<br>180.0 s, gap n/a | $\gamma = 0\%$<br>train/test = 0/0<br>180.0 s, gap n/a | $\gamma = 0\%$<br>train/test = 171/46<br>973.9 s, gap 2.8% |
| banknote | 4 | $\gamma = 0\%$<br>train/test = 215/40<br>180.4 s, gap 9.4% | $\gamma = 0\%$<br>train/test = 0/0<br>180.5 s, gap n/a | $\gamma = 0\%$<br>train/test = 311/74<br>180.6 s, gap 12.3% |
| wdbc | 2 | $\gamma = 40\%$<br>train/test = 28/5<br>180.2 s, gap 1.8% | $\gamma = 40\%$<br>train/test = 28/5<br>180.2 s, gap 1.8% | $\gamma = 40\%$<br>train/test = 38/8<br>180.2 s, gap 1.6% |
| wdbc | 3 | $\gamma = 20\%$<br>train/test = 41/9<br>1033.3 s, gap 0.4% | $\gamma = 40\%$<br>train/test = 0/0<br>180.4 s, gap 0.7% | $\gamma = 20\%$<br>train/test = 28/6<br>180.3 s, gap n/a |
| wdbc | 4 | $\gamma = 40\%$<br>train/test = 24/4<br>180.4 s, gap 1.3% | $\gamma = 0\%$<br>train/test = 0/0<br>180.8 s, gap 1.6% | $\gamma = 80\%$<br>train/test = 53/15<br>181.5 s, gap 10.2% |

Table 6: Best configurations of `main_iterative_algorithm.jl` on `banknote` and `wdbc`.

The clearest result in this section is the role of *shifts*. The $F_h^S$ variant with *shifts* is the most regular one: it reaches zero test error several times on `iris`, `seeds`, and `wine`, and also on `wdbc` at larger depths. The plain $F_h^S$ version remains useful but less stable. The exact version $F_e^S$ is rarely the best compromise; it is usually slower without a clear gain in test error. As in the grouped study, the short exploratory iterative runs on `titanic` and `diabetes` were not kept in the final comparison tables.

# 4 Additional Work

## 4.1 Feature Augmentation

A small feature engineering study was also carried out. The procedure builds sums, absolute differences, products, and ratios between variables, ranks the candidates by absolute correlation with the target, and then reruns an exact univariate tree at depth 2. For the larger datasets, the same idea is evaluated through precomputed augmented versions of `titanic` and `diabetes`.

| Dataset | Original data | Augmented version |
|---|---|---|
| iris | train/test = 5/1<br>split nodes = 2<br>11.7 s | train/test = 2/2<br>split nodes = 3<br>4.3 s |
| seeds | train/test = 11/2<br>split nodes = 3<br>60.1 s | train/test = 10/3<br>split nodes = 3<br>47.0 s |
| wine | train/test = 7/2<br>split nodes = 3<br>60.6 s | train/test = 5/2<br>split nodes = 3<br>60.0 s |
| titanic | train/test = 256/76<br>split nodes = 3<br>10.1 s | train/test = 182/39<br>split nodes = 3<br>10.3 s |
| diabetes | train/test = 3880/967<br>split nodes = 3<br>10.9 s | train/test = 3880/967<br>split nodes = 3<br>10.9 s |

Table 7: Effect of feature augmentation at depth 2. Each cell reports train/test errors, the number of active split nodes, and the solving time.

The effect remains limited. On `iris`, the generated variables improve the training fit but require one more active split node and slightly worsen the test result. On `seeds`, the change is small and does not reduce the tree size. On `wine`, the augmented version improves training error but keeps the same number of active splits. On `titanic`, the precomputed augmented dataset improves both train and test errors, but only within the same depth-2 tree size. On `diabetes`, nothing changes. This additional study therefore remains modest: simple extra variables can move the errors a little, but they do not produce smaller trees in these runs.

## 4.2 Opposite Corners, Linked Sets, and Rounding

The second additional study uses a dedicated univariate formulation at depth 2. In the exact formulation used elsewhere in the report, a sample only carries flow if it is correctly classified. This is not compatible with the equalities studied here, which assume a unit flow for every sample. The implementation in `main_q4_opposite_corner.jl` therefore uses three types of variables: a path-flow variable, a stopping variable, and a terminal assignment variable by class. Each training sample starts with unit flow at the root, follows exactly one root-to-terminal-node path, and the objective only counts assignments to the true class.

The linked-set equality can be justified with a short inductive argument. Consider a linked set $(P, Q, \bar{j})$. If a node splits on a feature different from $\bar{j}$, then each pair $(p_k, q_k)$ follows the same branch because

the two samples coincide on all the other coordinates. If the split uses $\bar{j}$, the values in $Q$ are a cyclic shift of the values in $P$, so any threshold sends the same number of samples from $P$ and from $Q$ to the left branch. Repeating the same argument down the tree shows that, for every predicted class, the total terminal assignment of $P$ and $Q$ must be identical. This gives the valid equality used in the model.

Three variants were compared: the unit-flow baseline, the same model with all equalities added from the start, and a cutting-plane variant that repeatedly solves the linear relaxation and adds only the violated equalities before solving the integer model. For the cutting-plane variant, the reported time is the total algorithm time, that is, the sum of all LP rounds and the final integer solve. On the original normalized data, no opposite-corner or linked-set equality is generated on `iris`, `seeds`, or `wine`. The useful comparison therefore starts from the first rounding level that creates nonzero candidates for each dataset.

| Dataset | Rounding | Mode | Added eq. | LP value | Train err. | Time (s) | B&B nodes |
|---|---|---|---|---|---|---|---|
| iris | 1 | baseline | 0 / 0 | 120.0 | 5 | 12.5 | 1606 |
| iris | 1 | all equalities | 237 / 4 | 120.0 | 5 | 8.1 | 317 |
| iris | 1 | cutting plane | 186 / 0 | 120.0 | 5 | 10.0 | 433 |
| seeds | 1 | baseline | 0 / 0 | 168.0 | 16 | 28.7 | 5409 |
| seeds | 1 | all equalities | 60 / 0 | 168.0 | 16 | 30.1 | 5406 |
| seeds | 1 | cutting plane | 44 / 0 | 168.0 | 16 | 30.7 | 5668 |
| wine | 0 | baseline | 0 / 0 | 142.0 | 21 | 15.3 | 139 |
| wine | 0 | all equalities | 1421 / 0 | 142.0 | 21 | 8.8 | 0 |
| wine | 0 | cutting plane | 1028 / 0 | 142.0 | 21 | 9.3 | 0 |

Table 8: Comparison of the unit-flow formulation, all equalities, and the cutting-plane variant on the first useful rounded version of each dataset. Added equalities are reported as opposite-corner / linked-set counts.

Two remarks stand out. First, the LP objective stays equal to the number of training samples in all tested cases. In practice, this means that the unit-flow relaxation remains weak on these datasets, and the added equalities do not change that value in the reported runs. Second, their effect on the search tree is mixed. On `iris`, the opposite-corner equalities clearly reduce the number of branch-and-bound nodes and the full model is the fastest one. On `seeds`, the generated equalities do not improve either the LP objective or the training error, and they slightly increase the total time. On `wine`, the equalities become useful only after a very coarse rounding, where they eliminate the search tree, but the rounded data are already much less faithful to the original geometry.

| Dataset | Rounding | Candidates | Added eq. | Train err. | Time (s) | B&B nodes |
|---|---|---|---|---|---|---|
| iris | none | 0 / 0 | 0 / 0 | 5 | 8.3 | 1223 |
| iris | 2 | 0 / 0 | 0 / 0 | 5 | 15.7 | 2170 |
| iris | 1 | 237 / 4 | 186 / 0 | 5 | 8.3 | 433 |
| iris | 0 | 1333 / 0 | 741 / 0 | 22 | 2.9 | 0 |
| seeds | none | 0 / 0 | 0 / 0 | 11 | 30.5 | 4921 |
| seeds | 2 | 0 / 0 | 0 / 0 | 11 | 30.4 | 4655 |
| seeds | 1 | 60 / 0 | 44 / 0 | 16 | 24.5 | 5663 |
| seeds | 0 | 2866 / 0 | 556 / 0 | 25 | 4.5 | 0 |
| wine | none | 0 / 0 | 0 / 0 | 5 | 30.4 | 3593 |
| wine | 2 | 0 / 0 | 0 / 0 | 8 | 30.5 | 3057 |
| wine | 1 | 0 / 0 | 0 / 0 | 7 | 20.8 | 2270 |
| wine | 0 | 1421 / 0 | 1028 / 0 | 21 | 8.8 | 0 |

Table 9: Effect of rounding with the cutting-plane strategy. Candidate and added equalities are reported as opposite-corner / linked-set counts.

The rounding study confirms that these equalities are highly sensitive to the data geometry. Mild rounding helps on `iris`: at one decimal, the training error is unchanged and the search tree becomes much smaller. The same behavior is not observed on `seeds`, where rounding hurts the training fit and does not simplify the branch-and-bound search. On `wine`, no useful equality appears before rounding to zero decimals, and this aggressive modification sharply degrades the training fit even though the model then solves at the root. Linked sets remain rare in practice: only `iris` at one decimal generates them, and none of these four linked-set equalities is violated by the LP solution. In this additional study, opposite-corner equalities are therefore the only ones that have a visible algorithmic effect.

# 5 Conclusion

The exact multivariate formulation remains the best practical reference on most datasets, especially at depths 2 and 3. The exact univariate formulation quickly becomes more expensive as depth increases, with more irregular behavior on `titanic` and `diabetes`.

The grouped formulation $F_U$ is useful as a univariate approximation on medium-sized datasets, and its multivariate version is also competitive on the five retained datasets except `banknote`. In the iterative part, the strongest variant remains $F_h^S$ with *shifts*: it is the one that most often gives the best compromise in test error on the retained runs.

The two additional studies remain interesting but secondary. Adding simple variables does not bring a general improvement, even after extending the analysis to `titanic` and `diabetes`, and it does not reduce the depth-2 tree size in the retained runs. The geometric equalities require a dedicated unit-flow formulation; on the original data they are almost absent, and after rounding their effect depends strongly on how much the data are altered. They can reduce the search tree on `iris` and on heavily rounded `wine`, but they do not improve the LP objective at depth 2 and they can easily damage the training fit. Overall, the clearest results still come from the comparison between the exact, grouped, and iterative formulations.